

HTTP Web Services & Model Serving

From MLflow champion to a production-shaped inference API

Lecture Overview

- Packaging & Versioning Models for Production
- Deployment Scenarios & Architectural Patterns
- Service Interfaces: REST vs gRPC
- Building APIs with FastAPI
- Production Inference Frameworks

Why Is This Important?

Shipping the model as a service is the real work:

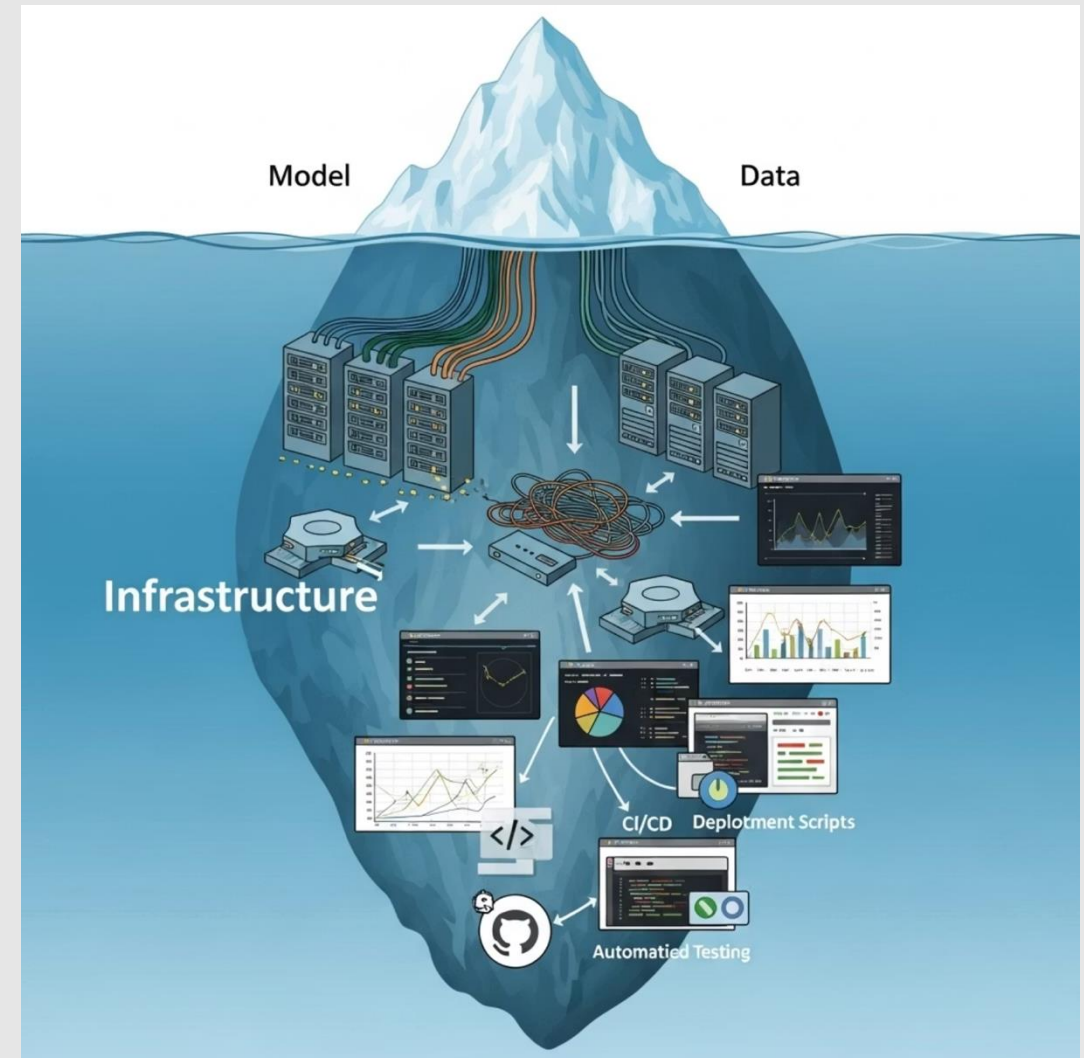
Session 6 — a stable /predict contract.

Session 8 — same contract, pretrained model.

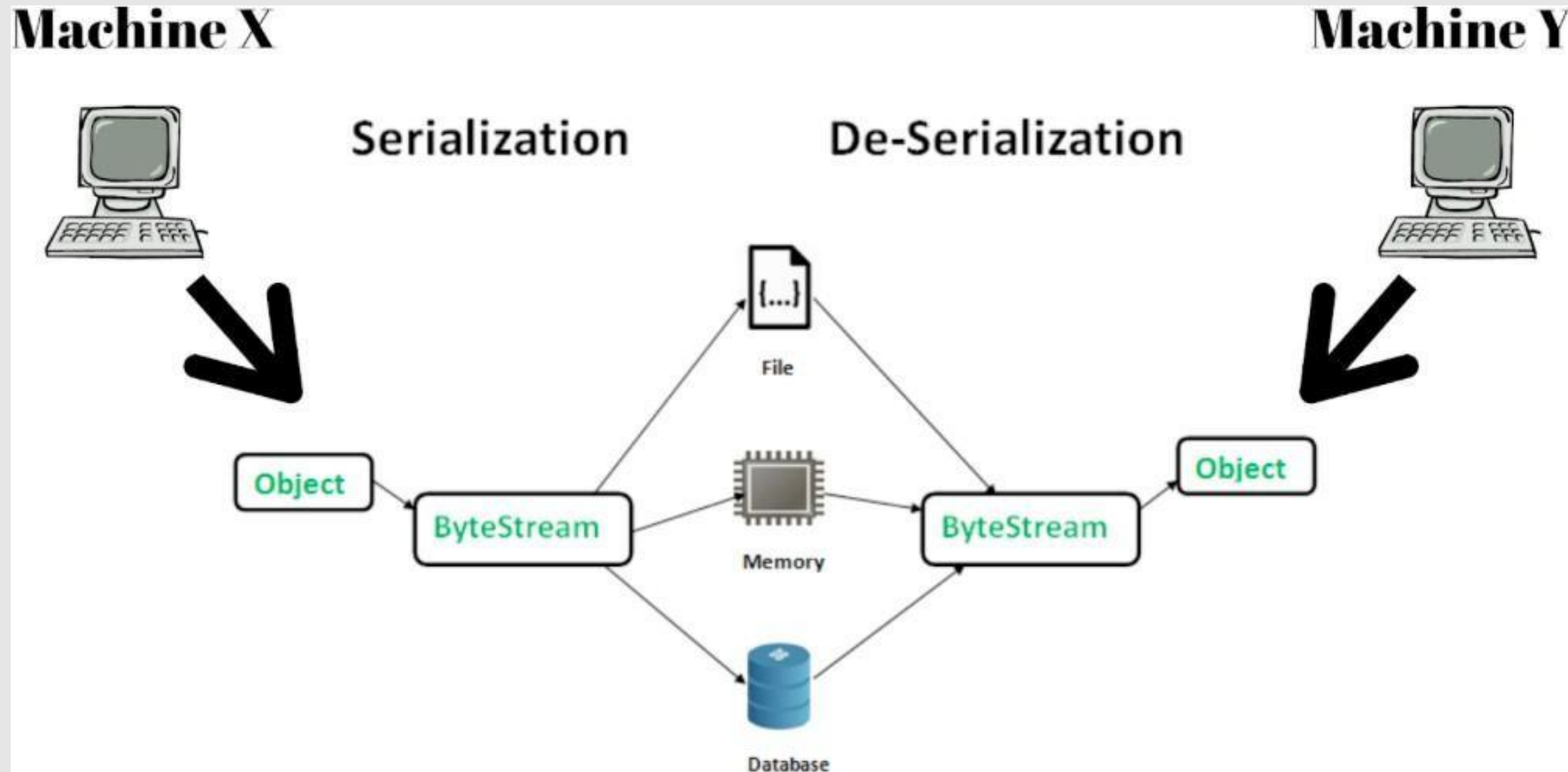
Session 11 — champion selected in MLflow.

Session 12 — champion as an HTTP service (today).

Session 13 — Grafana & cloud observability (next).



Model serialization



Pickle, Joblib, Cloudpickle

```
import joblib  
from my_module import SimpleModel
```

```
model = SimpleModel()
```

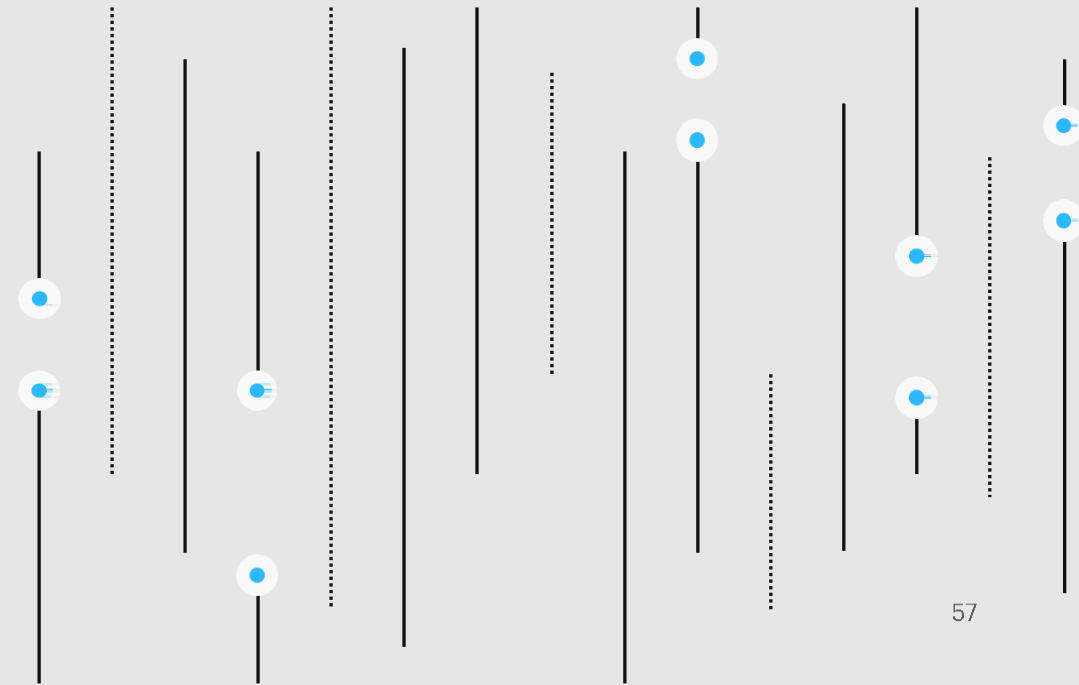
```
model_path = "model.joblib"  
joblib.dump(model, model_path)
```

```
loaded_model = joblib.load(model_path)  
print("Model loaded successfully")
```

Pickle, Joblib, Cloudpickle

- Easy
- **Insecure**
- You must save part of the code separately (move to modules)
- You must capture the runtime environment yourself

Security: pickle / joblib / cloudpickle are for trusted internal artifacts only. Loading an untrusted artifact can execute arbitrary code.



Framework-specific ways (PyTorch)

```
import torch
from my_module import SimpleModel

model = SimpleModel()

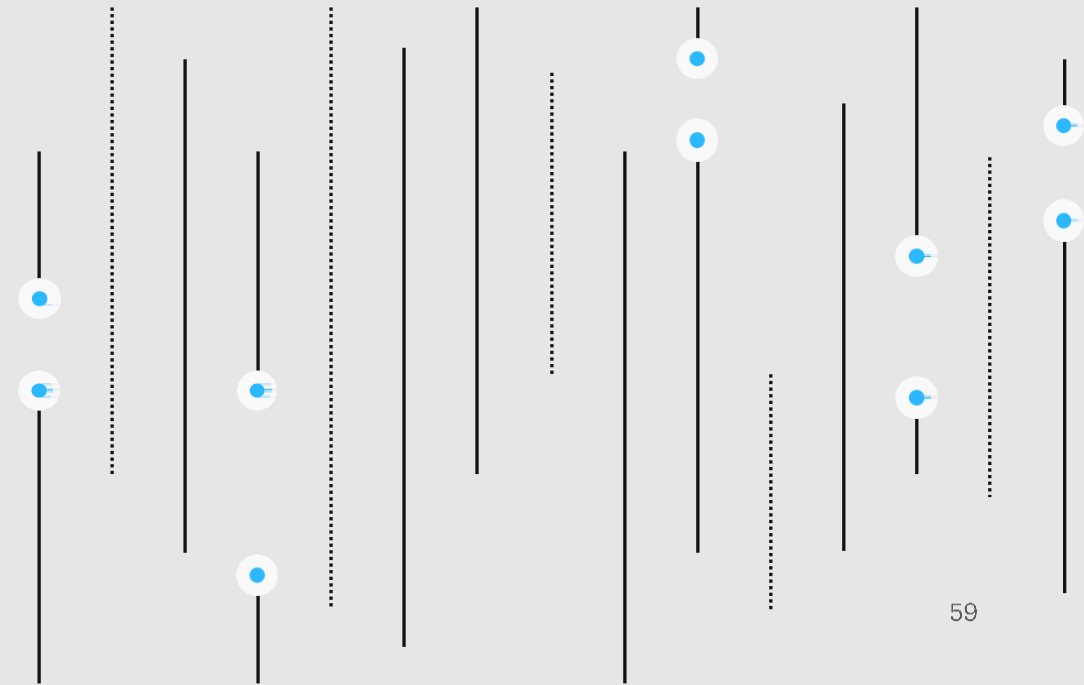
state = {
    "model_state_dict": model.state_dict(),
    "model_config": {"hidden_dim": 128},
}
torch.save(state, "checkpoint.pt")

checkpoint = torch.load(
    "checkpoint.pt", map_location="cpu", weights_only=True,
)

loaded_model = SimpleModel(**checkpoint["model_config"])
loaded_model.load_state_dict(checkpoint["model_state_dict"])
loaded_model.eval()
```

Framework-specific ways

- Available in almost every framework
- Not universal
- Part of the code and environment must still be saved separately



MLflow, Bento

```
import torch
import torch.optim as optim
import mlflow.pytorch
from my_module import SimpleModel

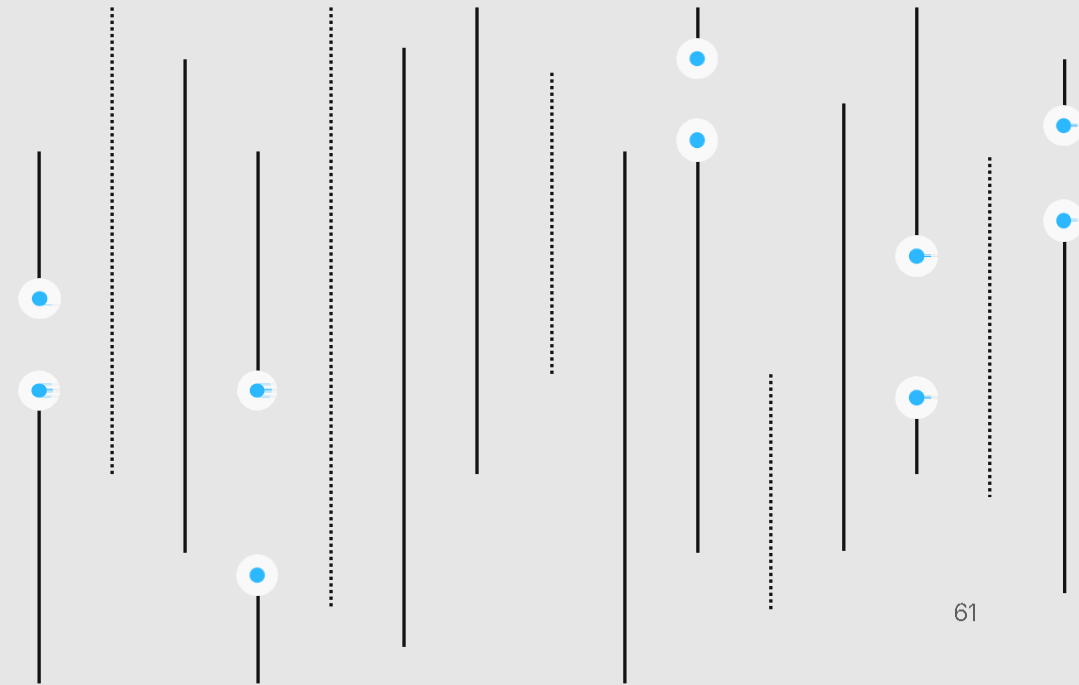
model = SimpleModel()
optimizer = optim.SGD(model.parameters(), lr=0.01)

mlflow.pytorch.save_model(model, "torch_model_mlflow") # Load the model

back
loaded_model = mlflow.pytorch.load_model("torch_model_mlflow")
```

MLflow, Bento

- Supports many ML frameworks
- Works well on the happy path; weaker for edge cases
- Can also save environment and pre/post-processing code



ONNX

```
import torch
from my_module import SimpleModel

model = SimpleModel()
model.eval()

example_input = torch.randn(1, 10)

torch.onnx.export(
    model,
    (example_input,),
    "model.onnx",
    input_names=["features"],
    output_names=["logits"],
    dynamic_shapes={"features": {0: "batch_size"}},
    dynamo=True,
    verify=True,
)
```

ONNX

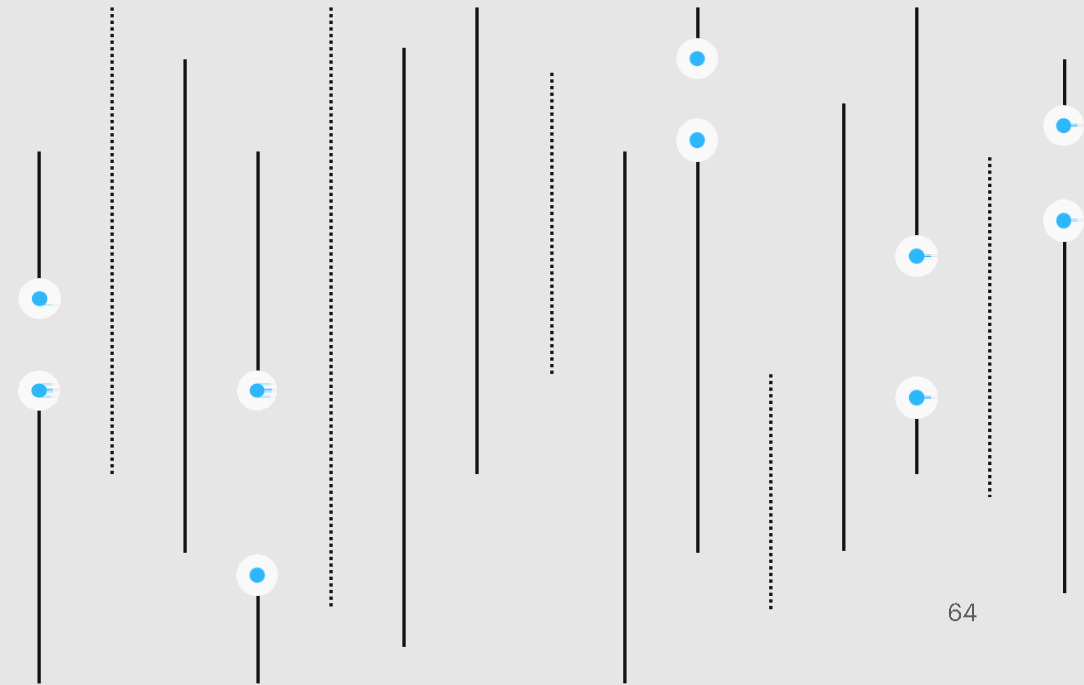
```
import onnxruntime as ort

session = ort.InferenceSession("model.onnx")
input_name = session.get_inputs()[0].name

outputs = session.run(
    None, {input_name: example_input.numpy()}
)
print(outputs)
```

ONNX

- Does not serialise everything
- Language-agnostic
- Optimised for inference



How to Choose a Serialization Method

Scenario	Better default
Quick internal sklearn prototype	joblib / pickle — trusted artifacts only
PyTorch checkpoint for retraining	state_dict + config + code version
Hugging Face model weights	safetensors + pinned revision
Portable tensor inference	ONNX / TensorRT / OpenVINO
Registry + signature + dependencies	MLflow Model
Edge / mobile PyTorch	torch.export / ExecuTorch (by target)

Should be done under the control of whoever is deploying (MLE), or per their instructions — e.g., saved to the Model Registry in the required way.

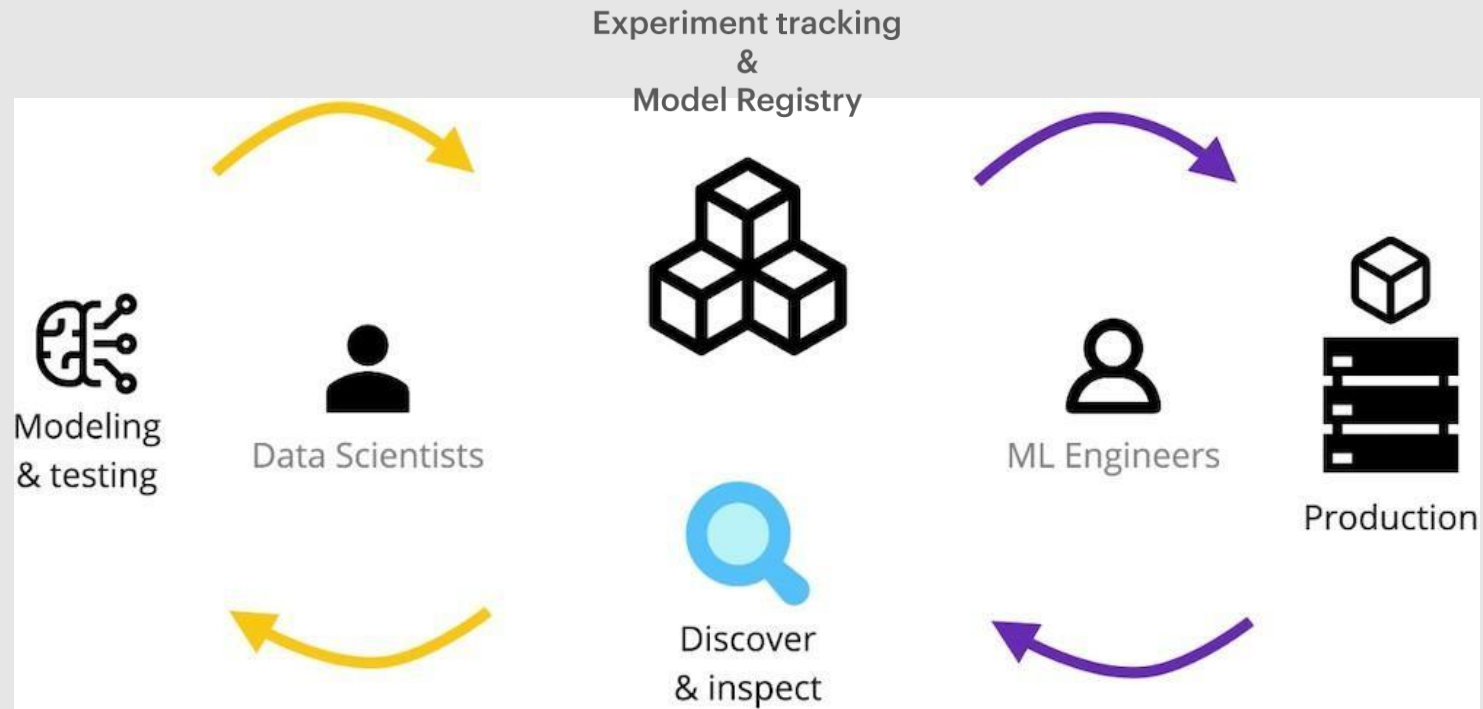
Preparing a model for deployment

- 01 **Weights & architecture.** Prefer safetensors; use pickle only for trusted internal artifacts.
- 02 **Code & preprocessing.** Loading logic, tokenizer, pre/post-processing, and the I/O schema.
- 03 **Environment & metadata.** Locked runtime deps (lockfile), config, version, and smoke tests.



Model Registry

Model Registry



Model Registry



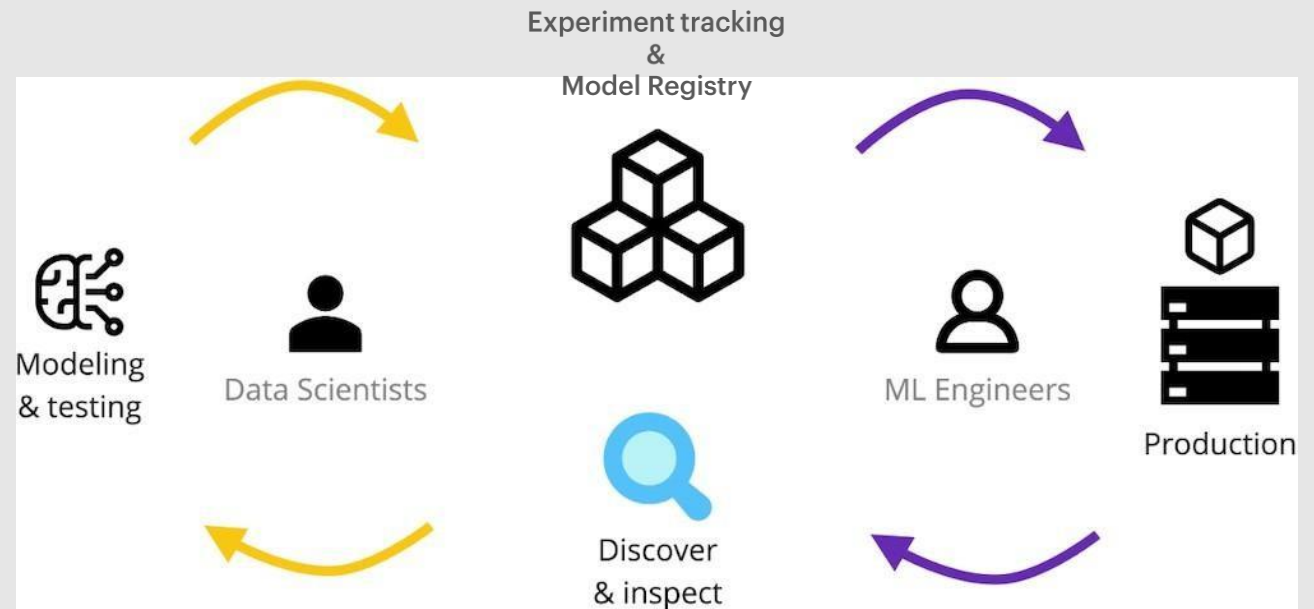
→ There can be one tool for tracking experiments:

mlflow

→ Or there may be several:

W&B + **mlflow**

mlflow +  **dockerhub**



Model Registry tasks

Lifecycle management:

- Store models (weights + deps)
- Version them & track changes
- Promote versions to production

Production history tracking:

- Investigate incidents during training: which code/data lead to the problematic model in prod?
- Investigate incidents in prod: which tests/monitoring did we miss?

Model sharing across teams:

- Search: find the right model, e.g., on HuggingFace
- Usage: load a model ready for inference

MLflow Tracking

```
import mlflow.sklearn
from mlflow import MlflowClient
from mlflow.models import infer_signature

signature = infer_signature(X_train, model.predict(X_train))

with mlflow.start_run():
    info = mlflow.sklearn.log_model(
        sk_model=model,
        name="classifier",
        signature=signature,
        input_example=X_train[:3],
        registered_model_name="bot_classifier",
    )

client = MlflowClient()
client.set_registered_model_alias(
    "bot_classifier", "champion", info.registered_model_version,
)
```

MLflow Model Registry



Registered Models >

mnist_model_prod

Created Time: 2023-09-25 12:10:09 Last Modified: 2024-01-19 10:35:06

> Description [Edit](#)

> Tags

▼ Versions [Compare](#)

New model registry UI ☒

Version	Registered at	Created by	Tags	Aliases	Description
✓ Version 8	2023-09-25 12:38:44		validation_status: pending ✎	@ challenger ✎	Uses decision tree
✓ Version 7	2023-09-25 12:38:41		validation_status: approved ✎	@ champion ✎	Uses a CNN based approach
✓ Version 6	2023-09-25 12:38:38		validation_status: approved ✎	Add	
✓ Version 5	2023-09-25 12:38:35		Add	Add	
✓ Version 4	2023-09-25 12:38:32		Add	Add	

↑
Versions of the same model

↗
Tags: free-form metadata & labels

↘
Aliases — shortcuts to the model, like `:latest` in Docker images.

MLflow Model Registry

The screenshot displays the MLflow Model Registry interface. At the top, the MLflow logo (2.10.0) and navigation tabs for 'Experiments' and 'Models' are visible. The 'Models' tab is active, showing a list of registered models. The model 'mnist_model_prod' is selected, with its creation and modification timestamps displayed. A modal dialog titled 'Add/Edit alias for model version 8' is open in the center. The dialog explains that aliases are mutable references to model versions and provides a search bar with two existing aliases: '@ challenger' and '@ new_alias'. A dropdown menu is open below the search bar, listing 'challenger' (marked 'This version'), 'champion' (marked 'Version 7'), and 'new_alias' (marked with a checkmark). The background shows a table of model versions (3-8) with columns for version number, registration time, validation status, and aliases. A 'New model registry UI' toggle is also visible on the right.

mlflow 2.10.0 Experiments Models

Registered Models >

mnist_model_prod

Created Time: 2023-09-25 12:10:09 Last Modified: 2024-01-19 10:35:06

> Description Edit

> Tags

▼ Versions Compare

Add/Edit alias for model version 8

Aliases allow you to assign a mutable, named reference to a particular model version. [Learn more](#)

@ challenger X @ new_alias X |

- challenger This version ✓
- champion Version 7
- new_alias ✓

Version	Registered at	validation_status	Aliases	Description
Version 8	2023-09-25 12:38:44	validation_status: pending	@ challenger	Uses decision tree
Version 7	2023-09-25 12:38:41	validation_status: approved	@ champion	Uses a CNN based approach
Version 6	2023-09-25 12:38:38	validation_status: approved	Add	
Version 5	2023-09-25 12:38:35		Add	
Version 4	2023-09-25 12:38:32		Add	
Version 3	2023-09-25 12:38:30		Add	

New model registry UI

MLFlow - getting the model via API

```
import mlflow
from mlflow import MlflowClient

# Load the champion by alias
model_uri = "models:/bot_classifier@champion"
model = mlflow.pyfunc.load_model(model_uri)

# Resolve concrete version for logs
mv = MlflowClient().get_model_version_by_alias(
    "bot_classifier", "champion"
)
version = mv.version
```

Pull method. The alias eases rollout — but log the resolved concrete version, not just the alias, for reproducibility.

How to choose a Model Registry

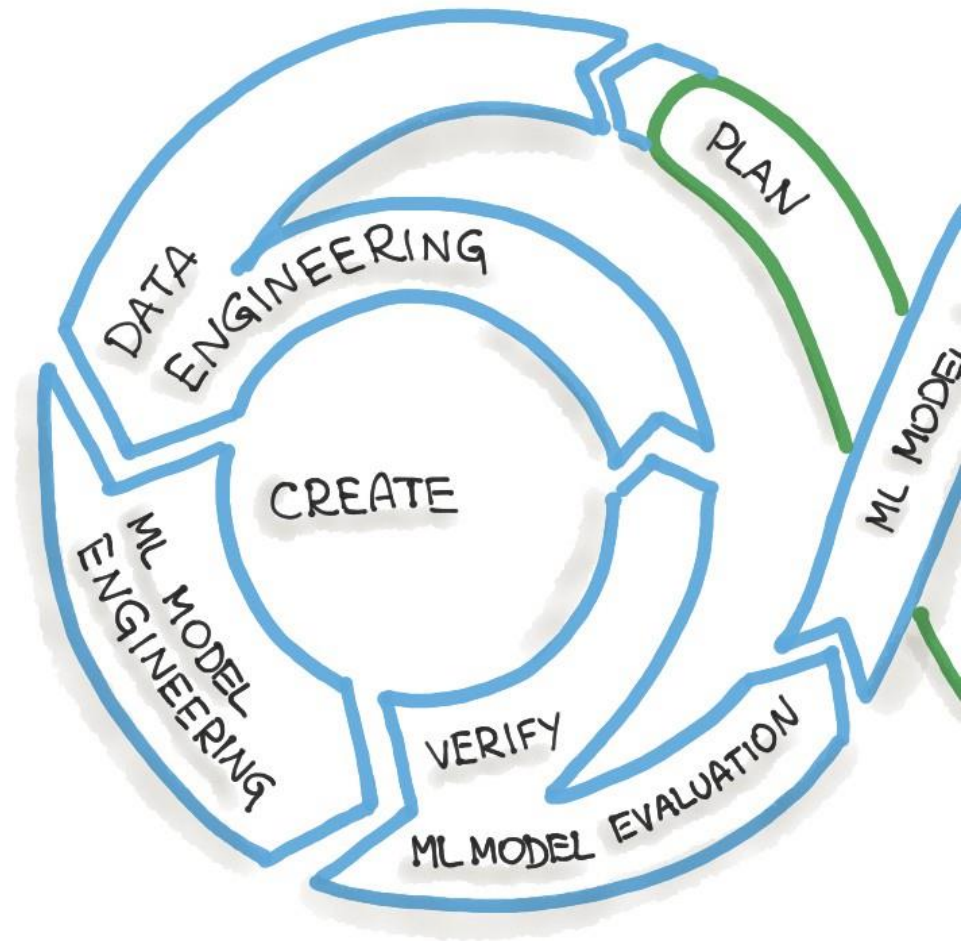
- 01 Which tasks do we solve?
- 02 What will deployment look like (push/pull)?
- 03 How much will it cost us?
- 04 What else exists in the ecosystem?

Model Lifecycle

So far we have:

- Trained several model versions.
- Chosen the best one (via MLflow experiments)
- Stored the best checkpoint in a registry (S3, MLflow, etc.)

What's next?

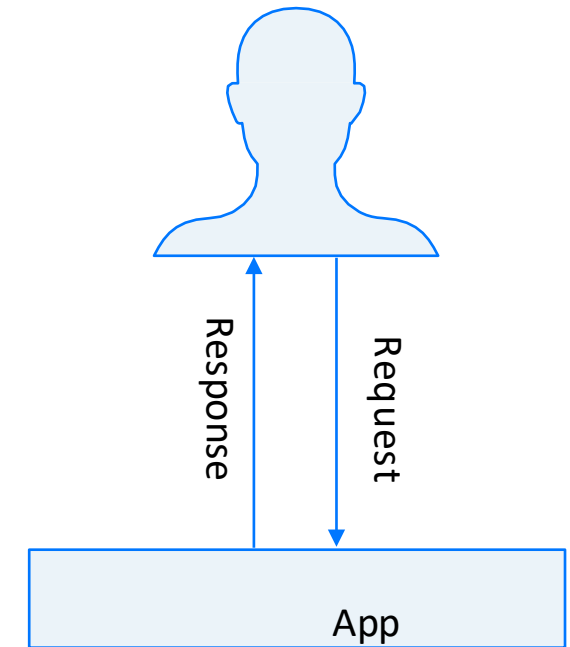


**MODEL
DEVELOPMENT**

Model Usage Scenarios

There are several most typical scenarios for using the model:

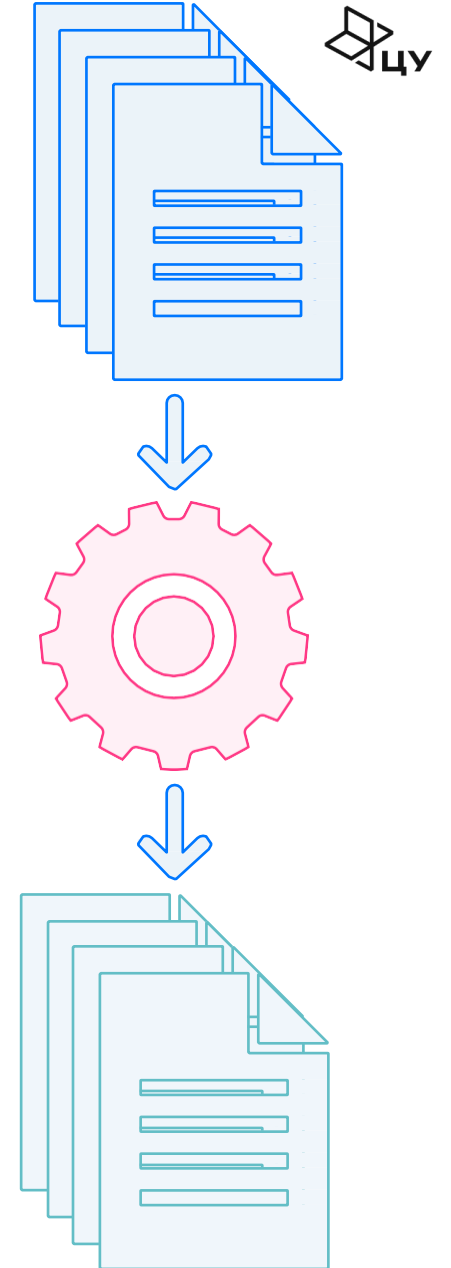
- 01 Request–Response** (real-time): the client sends data and receives a prediction.



Model Usage Scenarios

There are several most typical scenarios for using the model:

- 01 **Request–Response** (real-time): the client sends data and receives a prediction.
- 02 **Batch Scoring** (offline): periodically score a large data dump via cron/Airflow (tasks on the schedule)



Model Usage Scenarios

There are several most typical scenarios for using the model:

01

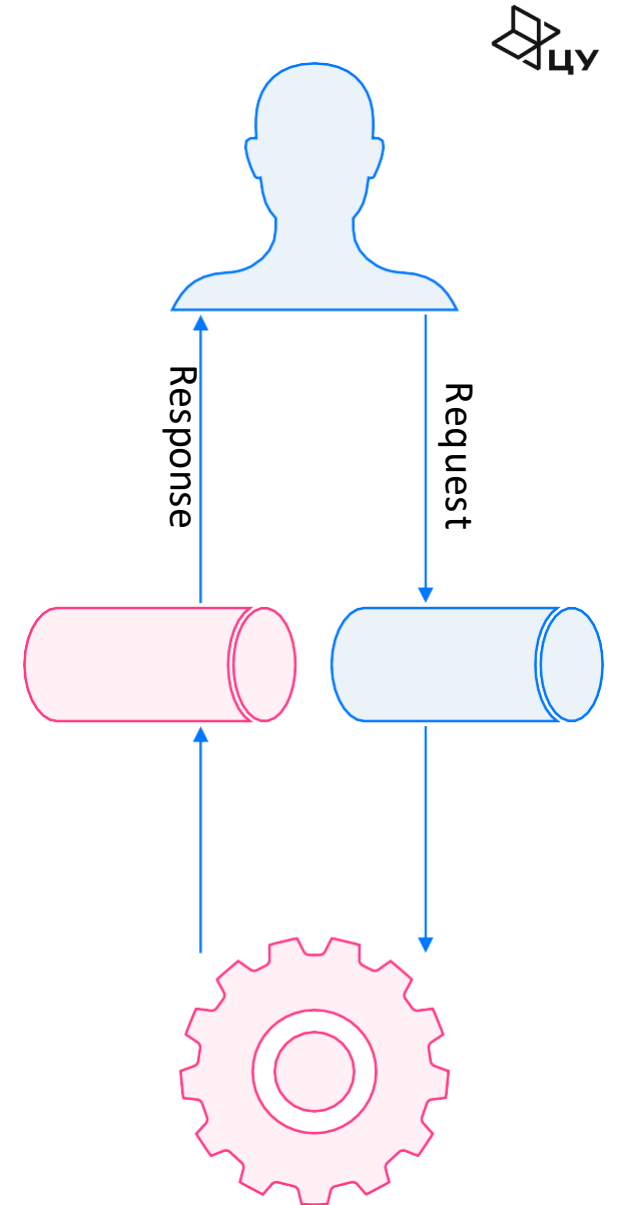
Request–Response (real-time): the client sends data and receives a prediction.

02

Batch Scoring (offline): periodically score a large data dump via cron/Airflow (tasks on the schedule)

03

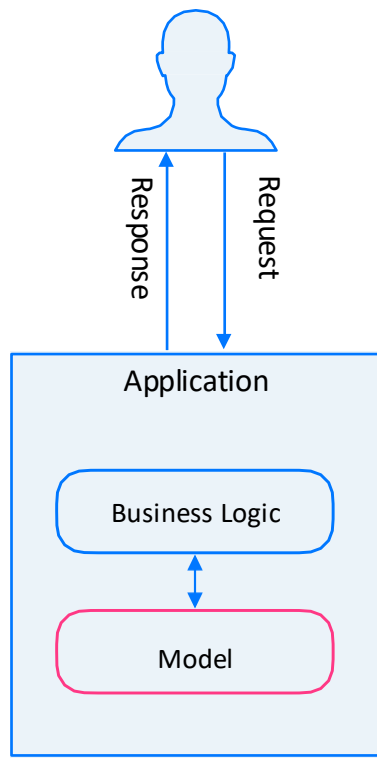
Streaming: tasks enter a queue for inference.



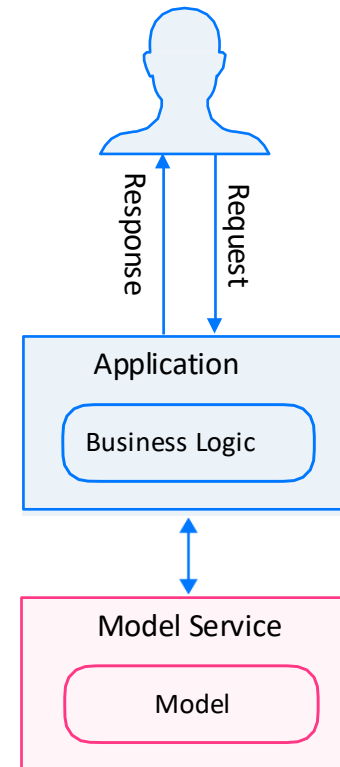
Integration Approaches

Today we will discuss only the request–response scenario.

How can a model be integrated into existing business logic from an architectural point of view?



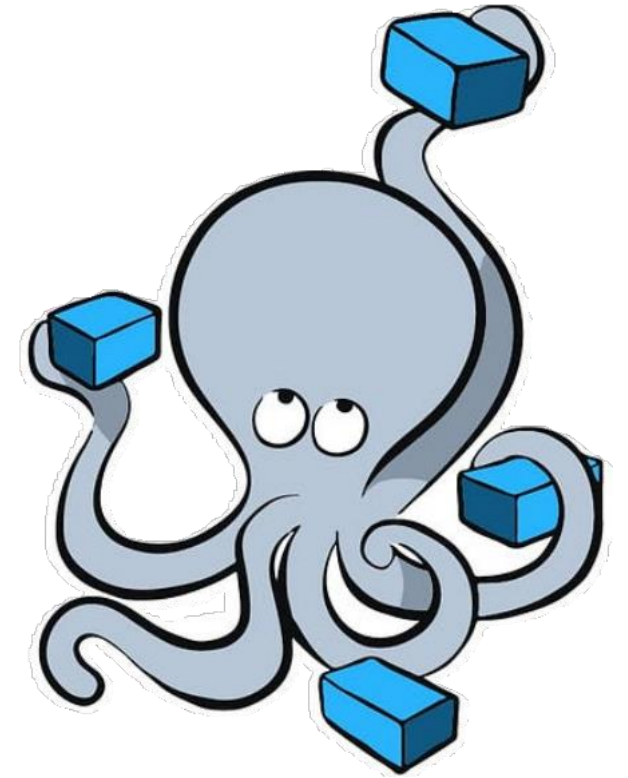
The model can be embedded directly in the existing application's code — a monolithic approach.



You can keep the call to the model inside the main application, while moving the model itself into a separate service (model as a service, MaaS) — a micro-service approach.

Docker compose

- **Docker Compose** is a tool for describing and running **multi-container applications**.
- It simplifies development and debugging of projects that consist of several components.
- To get started you write **one shared YAML file** describing all components and the relationships between them.



docker
Compose

Docker compose: commands

- For the commands to run correctly, one of the following service-configuration files must be present in the working directory:
 - compose.yaml
 - compose.yml;
 - docker-compose.yaml;
 - docker-compose.yml.
- To point Compose to a different file — or to **merge several configuration files** into one effective stack — use the **-f** flag

Bring up all services from compose.yaml

`docker compose up`

Same, but detached

`docker compose up -d`

Stop and remove services & networks

`docker compose down`

Tail logs of all services

`docker compose logs`

Tail logs of one service

`docker compose logs service_name`

List running services

`docker compose ps`

Restart services (ignores edits in compose.yaml)

`docker compose restart`

Pre-pull images defined in compose.yaml

`docker compose pull`

Docker compose: example

A minimal serving stack: your inference API plus an MLflow server, wired together for a local run.

```
# compose.yaml
services:
  inference-service:
    build: .
    environment:
      MODEL_URI: "models:/bot_classifier@champion"
      MLFLOW_TRACKING_URI: "http://mlflow:5000"
    ports:
      - "8000:8000"
    depends_on:
      - mlflow

  mlflow:
    image: ghcr.io/mlflow/mlflow:v3.1.0
    ports:
      - "5000:5000"
```

Docker compose: example

Bring the whole stack up with one command — and tear it down just as easily.

build images and start the stack

```
docker compose up --build
```

run detached, then follow logs

```
docker compose up -d
```

```
docker compose logs -f inference-service
```

stop and remove containers + network

```
docker compose down
```

Compose vs production orchestration

Compose is for local integration and reproducible demos — not your production runtime.

Production orchestration is a platform job:

- VM + systemd
- Cloud Run
- Amazon ECS
- Kubernetes / KServe
- Fly.io

Production Dockerfile with uv

FROM python:3.12-slim **AS** runtime

WORKDIR /app

RUN pip install --no-cache-dir uv

COPY pyproject.toml uv.lock ./

RUN uv sync --frozen --no-dev --no-install-project

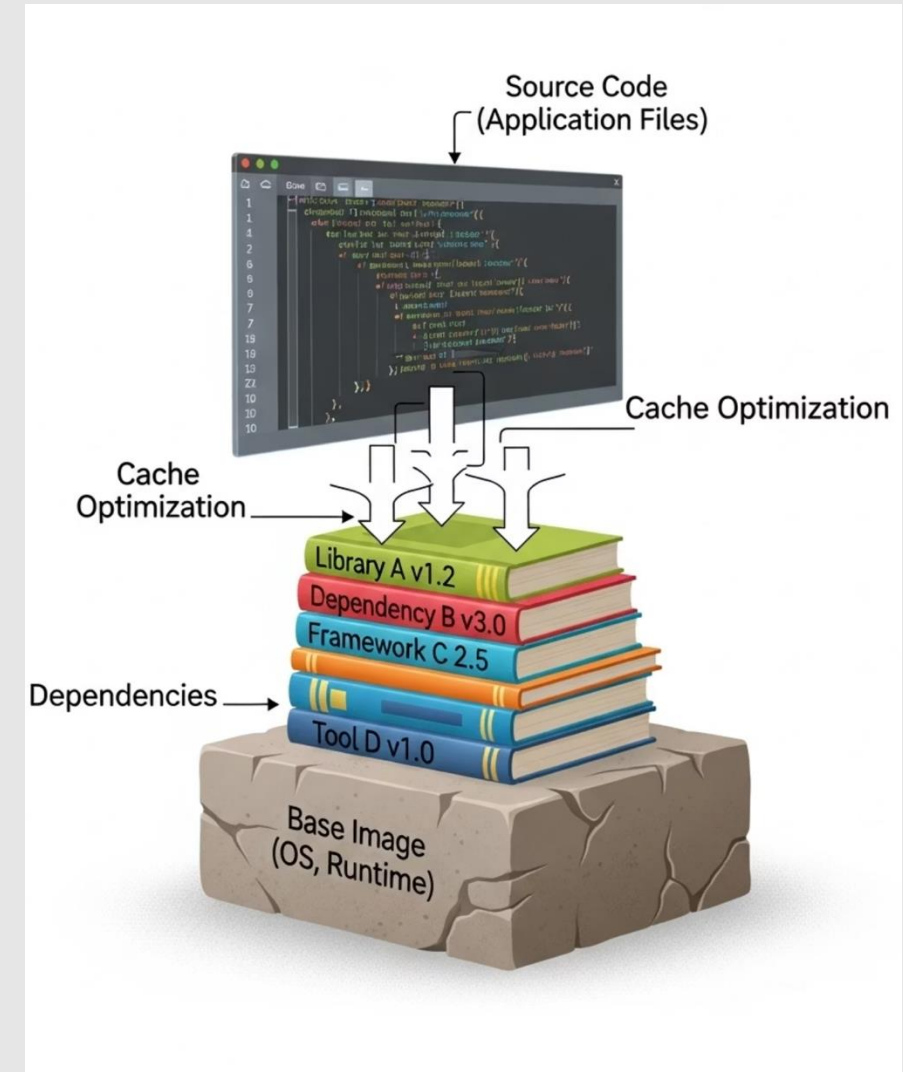
COPY src ./src

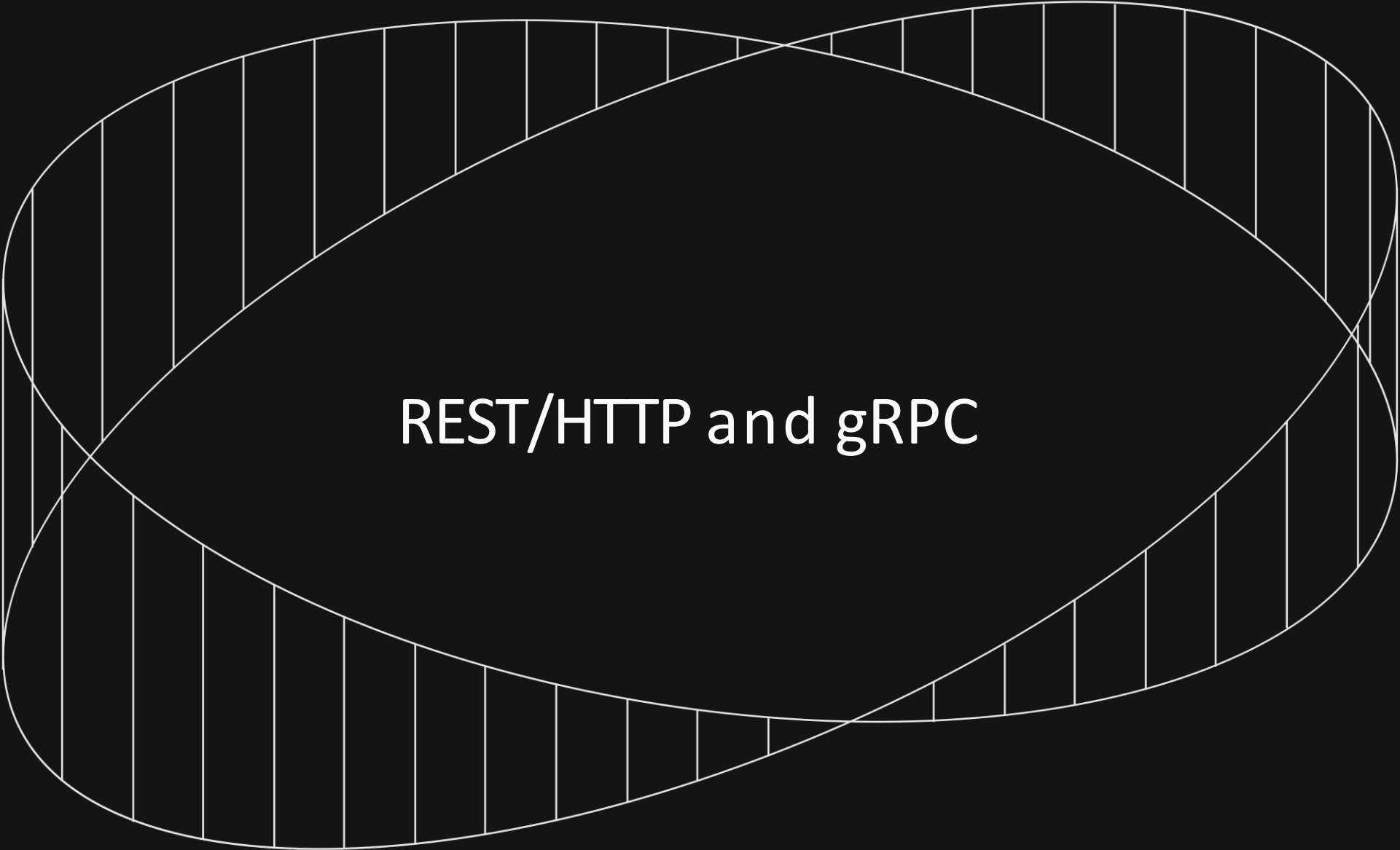
RUN useradd --create-home --uid 10001 appuser

USER appuser

CMD ["/app/.venv/bin/uvicorn", "src.main:app",
"--host", "0.0.0.0", "--port", "8000"]

1. Copy the lockfile before source code.
2. Install from the lockfile (uv sync --frozen).
3. No dev dependencies in the runtime image.
4. Run as a non-root user.
5. Never bake in .env, secrets, notebooks, raw data, or model caches.

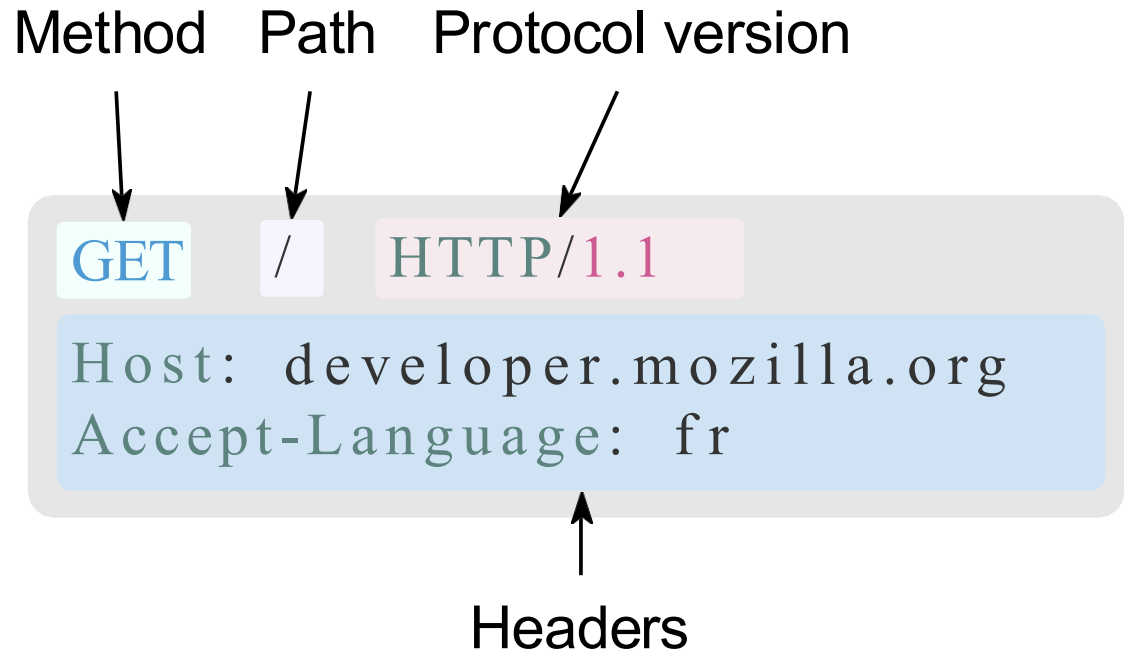


A Venn diagram with two overlapping circles. The interior of both circles is filled with vertical white lines of varying thickness, creating a striped effect. The central area where the two circles overlap is solid black.

REST/HTTP and gRPC

HTTP

- **HTTP (HyperText Transfer Protocol)** is an application-layer protocol used for communication between a client and a web service.
- For example, when you open a URL in a browser, the browser sends a **GET** request to the web server at the specified host.
- A request contains:
 - a *path* plus optional *headers*;
 - and, optionally, a *body* placed at the end of the request.

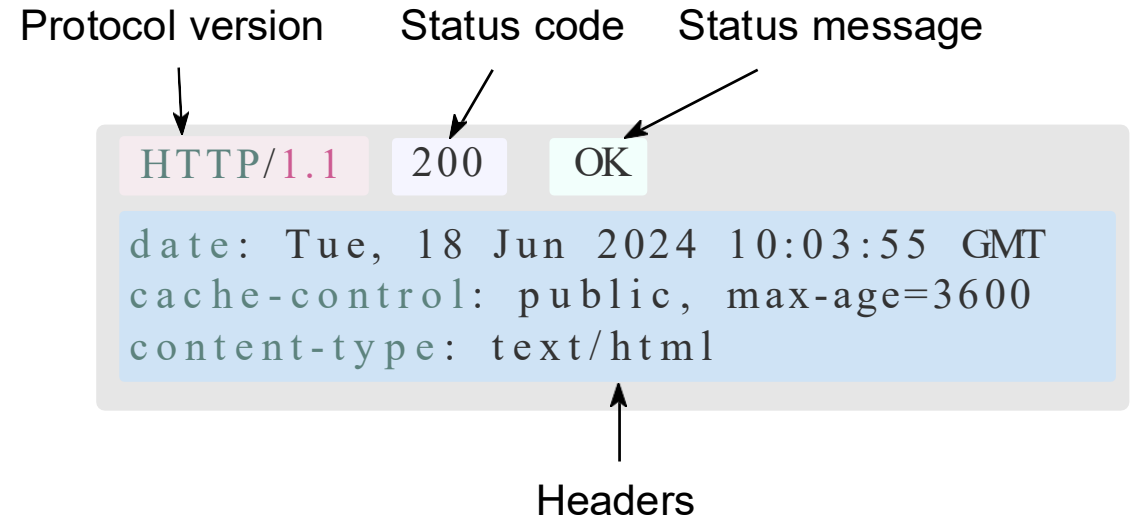


HTTP

The server answers with an HTTP response that includes:

- a status code and status message.
 - 1xx – informational
 - 2xx – success
 - 3xx – redirection
 - 4xx – client error
 - 5xx – server error

Headers and, optionally, a body (the HTML of a page).



HTTP CRUD

- HTTP has many verbs, but they boil down to four CRUD actions:
 - Create — add a resource (POST).
 - Read — fetch a resource (GET).
 - Update — modify a resource (PUT, PATCH).
 - Delete — remove a resource (DELETE).

REST

A RESTful API follows five key rules:

- **Client–Server** separation
- **Uniform Interface** (consistent endpoints)
- **Stateless** requests (each one is self-contained)
- **Cacheable** resources
- **Layered Architecture** (proxies, gateways, etc.)

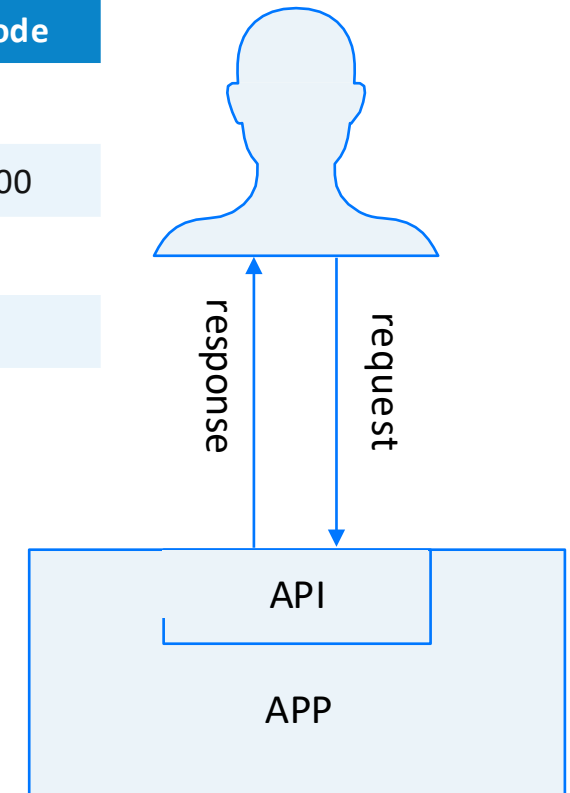
REST API

What Is a REST API?

A REST API is an architectural style, usually implemented over **HTTP(S)**:

- JSON payloads dominate in ML services — easy to inspect, validate, log safely, and test with curl;
- stateless, uniform and cacheable requests keep client and server decoupled.

Situation	Status code
Valid prediction	200
Invalid JSON / schema	422 / 400
Model not loaded / not ready	503
Upstream timeout	504
Internal crash	500



Protobuf

- Protobuf is a **binary, schema-based** data format.
- Like JSON, it is structured, but its binary form is **smaller and faster**.
- You must compile the **.proto** schema for every language you intend to use.

```
syntax = "proto2";

package tutorial;

message Person {
    optional string name = 1;
    optional int32 id = 2;
    optional string email = 3;

    enum PhoneType {
        PHONE_TYPE_UNSPECIFIED = 0;
        PHONE_TYPE_MOBILE = 1;
        PHONE_TYPE_HOME = 2;
        PHONE_TYPE_WORK = 3;
    }

    message PhoneNumber {
        optional string number = 1;
        optional PhoneType type = 2 [default =
PHONE_TYPE_HOME];
    }

    repeated PhoneNumber phones = 4;
}

message AddressBook {
    repeated Person people = 1;
}
```

Protobuf

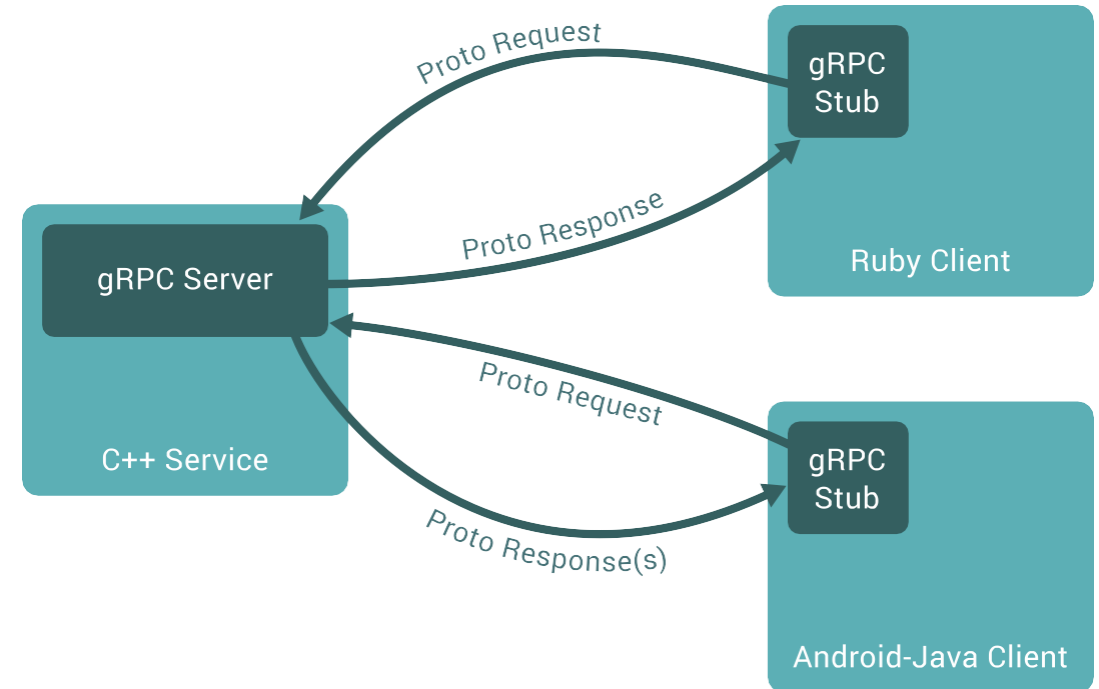
- **Protobuf** is a **binary, schema-based** data format.
- Like JSON, it is structured, but its binary form is **smaller and faster**.
- You must compile the **.proto** schema for every language you intend to use.

```
protoc --proto_path=$SRC_DIR --python_out=$DST_DIR $SRC_DIR/  
addressbook.proto
```

```
import addressbook_pb2  
person = addressbook_pb2.Person()  
person.id = 1234  
person.name = "John Doe"  
person.email = "jdoe@example.com"  
phone = person.phones.add()  
phone.number = "555-4321"  
phone.type = addressbook_pb2.Person.PHONE_TYPE_HOME  
  
with open("phone.pb", "wb") as f:  
    f.write(phone.SerializeToString())  
  
person_copy = addressbook_pb2.Person()  
with open("phone.pb", "rb") as f:  
    person_copy.ParseFromString(f.read())
```

gRPC

- **gRPC** (from Google) extends RPC for the cloud era.
- It lets you call remote procedures as if they were local functions.
- Data travels as Protobuf over **HTTP/2**, enabling streaming and multiplexing.



gRPC

Let's look at a simple example

- First, you need to define the request and response format, as well as the service itself, in the protobuf format.

```
syntax = "proto3";
```

```
package inference;
```

```
service Classifier {  
  rpc Predict (PredictRequest) returns (PredictResponse);  
}
```

```
message PredictRequest {  
  string text = 1;  
  string dialog_id = 2;  
  string id = 3;  
  int32 participant_index = 4;  
}
```

```
message PredictResponse {  
  string id = 1;  
  string message_id = 2;  
  string dialog_id = 3;  
  int32 participant_index = 4;  
  float is_bot_probability = 5;  
}
```

REST/HTTP vs gRPC

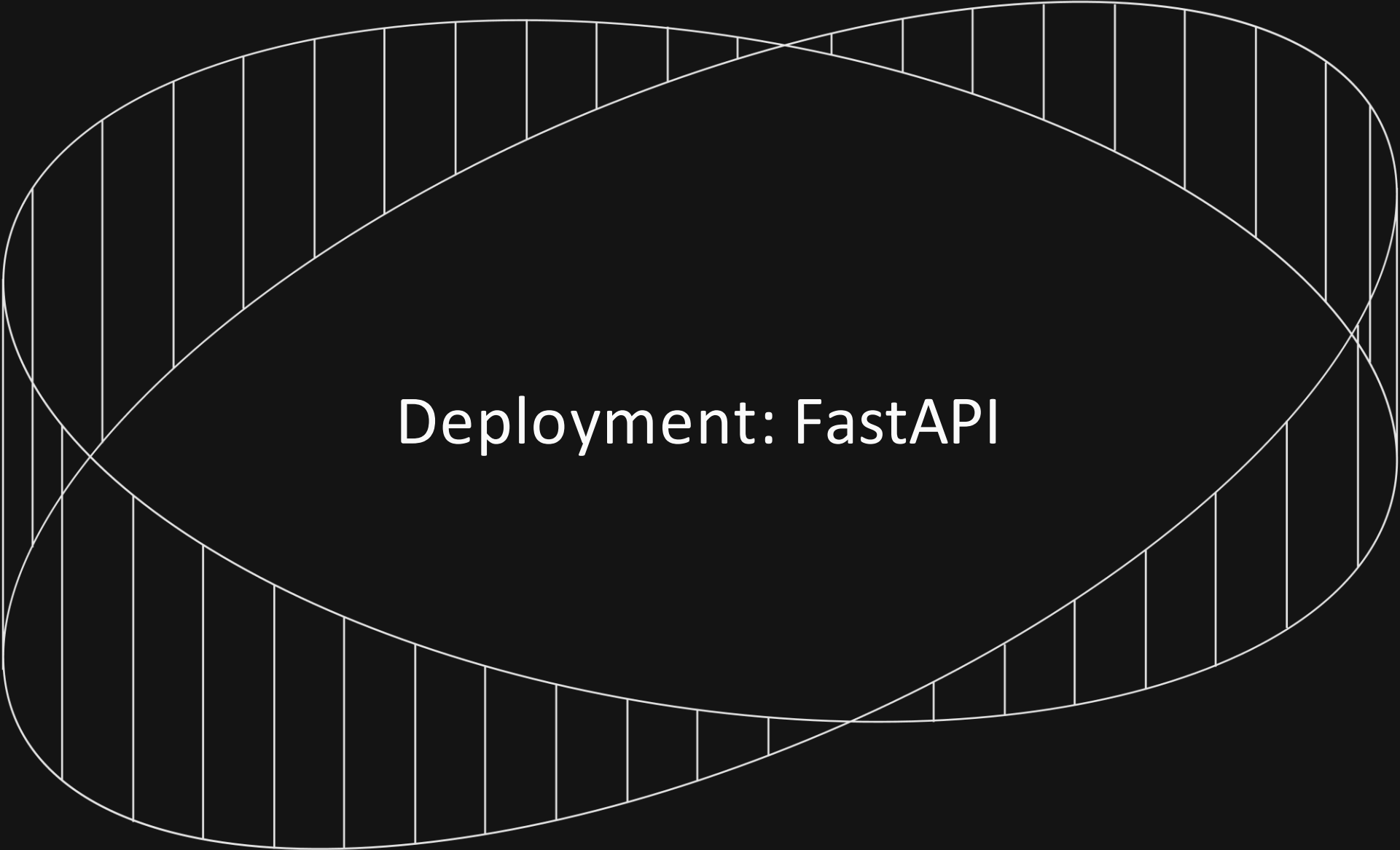
Use REST / OpenAPI when:

- Public or course-facing API
- curl / Swagger / browser tooling matters
- Schema is simple JSON
- Team is early-stage

Use gRPC when:

- Internal service-to-service
- Generated typed clients matter
- Strict binary schema matters
- High throughput / streaming matters

Not "faster, so better": gRPC wins for internal typed interfaces and streaming, at the cost of tooling and schema discipline.

A Venn diagram with two overlapping circles. The interior of both circles is filled with a pattern of vertical white lines. The intersection of the two circles is a solid black shape in the center, containing the text 'Deployment: FastAPI' in white.

Deployment: FastAPI

FastAPI: quick start

- We will need two python packages:
 - FastAPI.
 - Uvicorn.

```
pip install fastapi "uvicorn[standard]"
```

FastAPI: quick start

- We will need two python packages:
 - FastAPI.
 - Uvicorn.
- Let's throw together a simple application for demonstration.
- Let's launch a web service.
- Let's try to make a request.

```
from fastapi import FastAPI
```

```
app = FastAPI(title="Bot Classifier API")
```

```
@app.get("/health")
```

```
def health():
```

```
    return {"status": "ok"}
```

```
# run: uvicorn src.main:app --reload
```

FastAPI: Swagger

- Fast API automatically generates documentation for the API using Swagger UI.
- It is available at /docs.
- There is also an alternative version that is generated using ReDoc.
- It is available at /redoc.

The screenshot shows the Swagger UI for a FastAPI application. The browser address bar indicates the URL is `127.0.0.1:8000/docs`. The page title is "Fast API" with version "0.1.0" and "OAS3" specification. The API is named "default". The selected endpoint is `GET /items/{item_id}` with the description "Read Item Get". A "Try it out" button is visible. The "Parameters" section lists two parameters: `item_id` (integer, path, required) and `q` (string, query). The "Responses" section shows two responses: `200` (Successful Response) and `422` (Validation Error). The `200` response has a dropdown menu for the media type, currently set to `application/json`. The `422` response also has a dropdown menu for the media type, currently set to `application/json`. Below the responses, there is an "Example Value" section showing a JSON object:

```
{  "detail": [    {      "loc": [        "string"      ]    }  ]}
```

Pydantic

- Pydantic is a library for data validation.
- Tightly integrated with Python's built-in typing.
- The core logic is implemented in Rust, so it works quickly.

Let's look at an example of using Pydantic and FastAPI.

```
from uuid import UUID, uuid4
from pydantic import BaseModel, Field

class IncomingMessage(BaseModel):
    text: str = Field(min_length=1, max_length=5000)
    dialog_id: UUID
    id: UUID
    participant_index: int = Field(ge=0)

class Prediction(BaseModel):
    id: UUID
    message_id: UUID
    dialog_id: UUID
    participant_index: int
    is_bot_probability: float = Field(ge=0.0, le=1.0)
```

Essential API Endpoints

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model = load_from_mlflow()
    app.state.version = resolve_version()
    yield
    app.state.model = None

app = FastAPI(lifespan=lifespan)

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/ready")
def ready():
    return {"version": app.state.version}
```

```
@app.post("/predict",
          response_model=Prediction)
def predict(msg: IncomingMessage):
    p = app.state.model.predict_proba(msg.text)
    return Prediction(
        id=uuid4(),
        message_id=msg.id,
        dialog_id=msg.dialog_id,
        participant_index=msg.participant_index,
        is_bot_probability=p,
    )

@app.get("/metrics")
def metrics():
    return Response(generate_latest(),
                    media_type=CONTENT_TYPE_LATEST)
```

Bake-in vs Lazy-pull Model Strategy

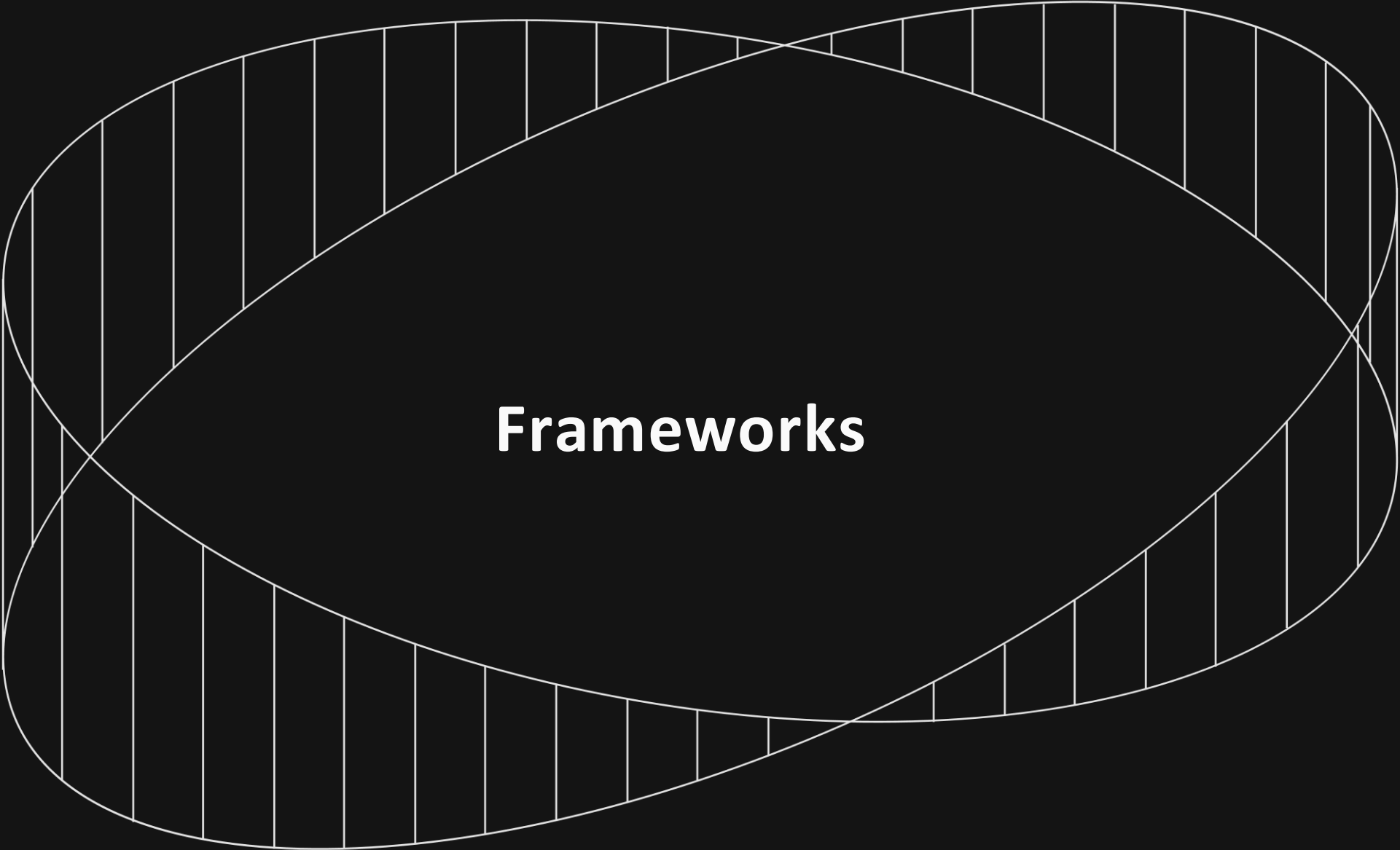
The trade-off is model-size and infrastructure dependent — measure image size, cold start, warm start, rollback time, and failure mode for your own model.

Strategy	Good for	Risk
Bake into image	Small, stable model; no startup network dependency	Large image; rebuild on every model update
Pull on startup	Frequent updates; registry-driven rollout	Cold start; registry outage; cache invalidation
Mounted cache / volume	Large models reused on the same VM / node	Cache consistency; cleanup; permissions
Init container / sidecar	Controlled Kubernetes-style loading	More moving parts

Problems of manual implementation

Manual FastAPI serving is flexible — but you own concurrency, batching, memory, timeouts, and GPU safety.

- `async def` does not make CPU/GPU-bound inference faster by itself.
- CPU-bound preprocessing can block the event loop.
- GPU inference may need a queue or semaphore to avoid VRAM OOM.
- For PyTorch inference, use `model.eval()` and `torch.inference_mode()`.
- Load the model once at startup — never inside `/predict`.
- Limit request size and input length.
- Track p50 / p95 / p99 latency, not just the average.
- Warm up the model after loading.



Frameworks

Frameworks for inference

- There are many frameworks for serving pre-trained models.
- For simple models they require almost **zero configuration**; for advanced cases they remain highly configurable.
- **The framework exposes an HTTP API** for you; scaling is usually handled by the deployment platform (Kubernetes, Cloud Run, ...).
- As the developer you usually specify only:
 - **Input / output schema**
 - **Dependencies**
 - Optionally — any **pre-processing, post-processing, or custom inference logic**



NVIDIA

TRITON INFERENCE SERVER



MLflow Models Serve

```
mlflow models serve \  
  -m "models:/bot_classifier@champion" \  
  --env-manager local --port 5000
```

The PyFunc interface provides a consistent contract across different frameworks, making it easy to switch between model types without changing your serving code.

NVIDIA Triton

- To serve a model with Triton you need to:
 - **Convert or trace** the checkpoint into **ONNX** (or another format Triton supports).
 - Assemble a **model repository** – each model gets a folder containing its config.pbtxt plus one or more version sub-folders with the actual weights.
 - Launch the Triton container and point it at that repository; the server will start exposing gRPC/HTTP endpoints automatically.



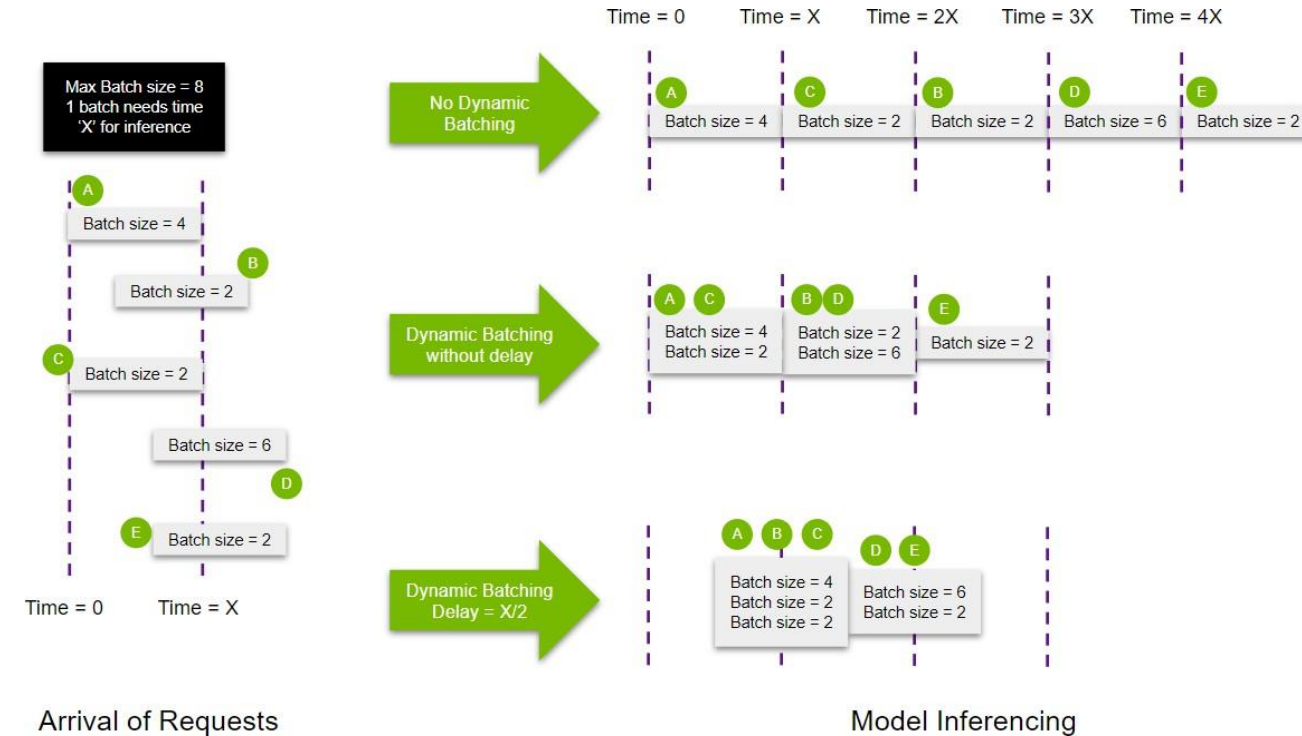
Model repository

- A *model repository* is a directory tree Triton scans for models and their configs.
- Each top-level folder is one model, with version sub-folders (1, 2, 3 ...) for the weights.
- In simple cases Triton can auto-complete parts of the model configuration.
- In production, write **config.pbtxt** explicitly: inputs, outputs, batching, instance groups, version policy.

```
model_repository/
├── text_detection
│   ├── 1
│   │   └── model.onnx
│   ├── 2
│   │   └── model.onnx
│   └── config.pbtxt
└── text_recognition
    ├── 1
    │   └── model.onnx
    └── config.pbtxt
```

Dynamic batching

- **Dynamic batching** groups incoming requests into a single batch.
- It primarily improves **throughput** and hardware utilization.
- It can improve latency under load — but aggressive queue delay hurts **p99 latency**.
- Key params: **max_batch_size** and **max_queue_delay_microseconds**.
- Typically for stateless models; pairs well with multiple model instances (GPUs / servers).



Inference Framework Comparison

Category	Tools	What they solve
Custom HTTP service	FastAPI	API contract, custom logic — our course service
Model package / demo serve	MLflow Models Serve	Quick validation of a registered model
Tensor / GPU inference runtime	NVIDIA Triton	ONNX/TensorRT/Python backend, batching, model repo
LLM serving runtime	vLLM , TGI , SGLang , llama.cpp	Generation, KV-cache, OpenAI-compatible APIs
Python service packaging	BentoML	Service abstraction and packaging
Kubernetes serving platform	KServe , Seldon , Ray Serve	Rollout, autoscaling, routing, canary
Legacy PyTorch serving	TorchServe	Legacy — limited maintenance

Lecture Takeaways

- A checkpoint is not a deployable package — ship weights, code, preprocessing, config, dependencies, schema and tests.
- MLflow Registry aliases/tags select the champion; always log the resolved concrete version.
- REST / OpenAPI suits course-facing APIs; gRPC suits internal typed interfaces and streaming.
- A production-shaped FastAPI service has lifespan, /health, /ready, /predict and /metrics — on the stable /predict contract.
- Inference runtimes (NVIDIA Triton, vLLM) are not the same as serving platforms (KServe, Ray Serve).

Practice / Homework Assignment

Goal: serve the MLflow champion behind the course /predict API.

- Load models:/bot_classifier@champion from MLflow once at startup (FastAPI lifespan).
- Preserve the real IncomingMessage → Prediction schema.
- /health returns 200; /ready returns the model name and resolved version.
- /predict returns is_bot_probability in [0, 1].
- Tests: valid request → 200, invalid UUID → 422, probability in range.
- Log request_id, model_name, model_version, latency_ms, status_code.
- Do not log raw user text by default.
- Measure p50 / p95 latency over 20 repeated requests.
- Add a smoke-test curl command to the README.
- Add a Dockerfile and compose.yaml for local run.